The Computational Boundary of a Fixed Transformer: Formal Inference, Unification, and Competence-Gated Rule-Like Computation

Paper 0.7 — Formal Computation Boundary Research Line

Integrated v3 protocol — August 2026

Abstract

We study the boundary between representing a formal structure, learning it from examples, and executing it systematically. A controlled ladder begins with propositional identities and modus ponens, then varies proof-chain depth, branching, distractors, and multi-premise rules. A second ladder begins with atomic matching and variable binding, then varies functor depth, arity, repeated-variable consistency, substitution propagation, and first-order proof depth. Every example has exact symbolic ground truth, minimal proof/term complexity, and disjoint-symbol controls. Fixed one-shot Transformers are evaluated over model depth, width, heads, and training budget; frontiers are reported only at measured competent points. The earlier Paper 0.7 proposal for fuzzy functors and learned semantic type dispatch is retained as a downstream application rather than the primary instrument. Paper 0.6 now supplies competent bounded composition but still fails to establish seed-stable depth-general predicates, so semantic rule-dispatch claims remain gated. The integrated program asks where one-shot execution fails and explicit iteration becomes a Paper 0.9 comparison hypothesis.

1 Central distinction and scope

Papers 0.5 and 0.6 motivate

$$\boxed{\text{representability} \neq \text{learnability} \neq \text{systematic executability.}}$$

A residual state may encode a relation that the model does not use correctly; a model may fit bounded examples without learning a procedure; and a learned procedure may execute only within a finite complexity range. Paper 0.7 therefore replaces broad claims that Transformers "reason" with exact competence surfaces.

The primary comparison machine is a fixed, one-shot causal Transformer. Autoregressive scratchpads, recursive interpreters, unification engines, and agent/tool loops belong to Paper 0.9. Natural-language logic benchmarks are optional external validity only because their proof structure, contamination, and lexical shortcuts are not exactly controlled.

2 Experimental standard

For task family P and formal complexity vector $\mathbf{C}$, measure

$$A(P, \mathbf{C}; L, W, H, T),$$

where L, W, H are layers, width, and heads, while T records optimizer updates, examples, and tokens. Joint sweeps define a measured Pareto competence region because $\mathbf{C}$ has no total order. Axis-specific frontiers vary one axis while all others are fixed or matched; they are never extrapolated beyond tested values. The primary gate is .80 accuracy and the secondary gate .90, relative to explicit chance/shortcut baselines. A central cell passes only if every one of at least three model seeds passes after aggregation over at least three crossed generator seeds; bootstrap intervals, evaluation count, mean, worst cell, and variance are reported.

Generalization splits separately test new symbol identities, new formulas at trained complexity, greater proof depth, greater term depth, and unseen combinations of templates. Sequence length is controlled independently. Propositional targets are conclusion symbols or truth values scored by exact match. Unification targets are canonical substitution sequences scored by exact match plus variable/component accuracy; negative cases require an explicit failure label. Every row stores its proof, minimal depth, canonical target, and shortcut audit.

3 Part I: propositional computation ladder

3.1 Local and one-step rules

P0 contains double negation, conjunction elimination, and disjunction introduction:

$$\neg\neg A \Rightarrow A, \qquad A \wedge B \Rightarrow A, \qquad A \Rightarrow A \vee B.$$

P1 is modus ponens, $A, A \Rightarrow B \vdash B$. P2 separates modus tollens, $A \Rightarrow B, \neg B \vdash \neg A$, from contraposition as a rule transformation, $A \Rightarrow B \mapsto (\neg B \Rightarrow \neg A)$. Trained and held-out forms are labeled separately.

3.2 Proof chains and branching

P3 fixes chain length two. P4 varies

$$\{A_i \Rightarrow A_{i+1}\}_{i=0}^{C-1}, \quad A_0 \vdash A_C, \qquad C \in \{1, 2, 3, 4, 6, 8, 12, 16, 24, 32\}.$$

where C is the number of inference edges in a minimal proof. Training is bounded at C_{train} and testing extends beyond it. P5 independently varies proof depth, branching, irrelevant clauses, competing conclusions, and multiple proof paths. P6 introduces multi-premise rules such as $A \wedge B \Rightarrow C$ before controlled nested Boolean fragments.

The primary propositional outputs are accuracy, target rank/margin, first and stable decision depth, extrapolation gap, and $C_{\text{prop,max}}$. An $L \times C$ interaction establishes a depth-resource effect, not serial execution; iterative computation additionally requires ordered intermediate-state and causal evidence.

4 Part II: first-order unification and inference

The unification ladder isolates structural requirements before combining them with proofs:
F0. atomic equality and incompatibility;
F1. variable binding, $P(x)$ with $P(a) \Rightarrow x = a$;
F2. unary functors, $P(f(x))$ with $P(f(a))$;
F3. arity two and three;
F4. nested functors at depths $\{1, 2, 3, 4, 6, 8, 12\}$;
F5. repeated-variable consistency, where $f(x, x)$ cannot unify with $f(a, b)$ unless $a = b$;
F6. substitution across atoms, $P(x), Q(x)$ with $P(a), Q(a)$;
F7. first-order modus ponens;
F8. predicate chains with proof depth varied independently of term depth;
F9. rules containing structured terms, such as $P(f(x)) \Rightarrow Q(x)$.

Complexity axes are proof depth, functor depth, arity, variable count/reuse, branching, distractors, vocabulary, and serialization length. Term depth is the maximum number of functor edges from root to leaf. Successes store a canonical most-general unifier modulo deterministic variable renaming and require an occurs check. Balanced failures store a symbol, arity, occurs-check, or repeated-variable-conflict certificate. Repeated-variable failures are as important as successes; F6 substitution propagation is scored separately from F8 proof accuracy.

5 Architecture and training design

The planned grid is

$$L \in \{1, 2, 4, 6, 8, 12, 16, 24\}, \quad W \in \{16, 32, 64, 128, 256\}, \quad H \in \{1, 2, 4, 8\}, \quad T \in \{1, 2, 4\} \times .$$

The first stage uses a designed factorial subset with deeper/narrower and shallower/wider controls. Exact total and non-embedding parameters, head/FF dimensions, optimizer, tokens, examples, and wall time are recorded; matches are labeled approximate when exact equality is impossible. Full curves expand only along axes that move a frontier. Learning curves distinguish undertraining, unstable optimization, converged interpolation, and extrapolative competence.

Resource hypotheses are tests, not architectural labels: layers may move proof/term depth, width may move vocabulary/state capacity, and heads may move branching/selection. A claim requires the corresponding interaction under matched parameter and training controls.

5.1 Executable dataset and phase-plan checkpoint

The formal protocol now has an executable, deterministic CPU-side dataset layer. It serializes P0–P6 and F0–F9 into canonical JSONL records carrying separate fields for proof depth, term depth, arity, variable count and reuse, branching, distractors, and symbol namespace. Train, validation, and test use distinct namespaces; the validator rejects duplicate record identifiers or namespace overlap. Proposition labels are checked against exact forward closure and stored minimal proof depths. Unification labels and canonical substitutions are recomputed by the exact engine, including occurs-check, arity, symbol, and repeated-variable failures where applicable.

A CPU smoke materialization contains 252 validated rows: 140 train, 56 validation, and 56 test rows across all 17 stages. Its phase-plan smoke has 48 cells, crossing the two task families with $L \in \{1, 2\}$, $W \in \{16, 32\}$, $H \in \{1, 2\}$, one training budget, and three model seeds. The tracked full materialization contains 22,400 validated rows and enumerates 2,880 planned cells over the complete grid. These counts validate serialization, split construction, exact labels, and run enumeration only. They are *not* trained-model results and supply no evidence of Transformer competence or a formal-computation frontier; accelerator runs remain pending.

6 Competence-gated mechanism experiments

Only competent cells receive layerwise proof-state/target probes, SA/FF ablation, head utility, role-matched replacement, and selected token-level Q/K/V tracing. A mechanism comparison uses the smallest competent cell, the immediately smaller incompetent cell, and a matched parameter-allocation control. Final accuracy alone cannot establish serial proof execution, pointer jumping, variable binding, or unification.

For a target y, diagnostic logits and stable depth are

$$z_\ell = W_U \,\mathrm{LN}_f(r_\ell), \qquad L_{\mathrm{stable}} = \min\{\ell : \arg\max z_j = y \ \forall j \geq \ell\}.$$

Causal interventions report target-rank/margin damage, top-1 flips, and JS divergence. Attention motifs remain descriptive until token masking, Q/K/V patching, or head/component intervention agrees.

7 Downstream application: fuzzy functors and semantic dispatch

The original Paper 0.7 proposal studies transformations $F(x) \to y$ whose behavior depends on a learned semantic type. Its hypotheses remain useful:

- substitution-tolerant functor invariance;
- category-conditioned dispatch to different continuation families;
- transfer to unseen category members and member–functor combinations;
- state-conditioned residual rewriting, $r_{\ell+1} = r_\ell + F_\ell(r_\ell)$;
- uncertainty restructuring over a rule-consistent continuation set.

This is now a downstream application of the formal ladder. Formal tasks first establish whether binding, substitution, and composition are learned under exact controls; only then may fuzzy text-based analogues be interpreted.

Its executable dataset is a controlled textual world with instance–subcategory–category–supercategory relations, overlapping observable properties, multiple lexical realizations, and paraphrases. Unary $F(x)$, binary $R(x, y)$, nested $G(F(x))$, defaults, exceptions, and competing continuations are introduced without exposing latent type labels as privileged supervision. Held-out cells cross member identity, member–functor pairing, paraphrase, composition length, and category-boundary/exception cases. At pre-SA, post-SA, and post-FF states, decode $p_\ell = \mathrm{softmax}(W_U \,\mathrm{LN}_f(r_\ell))$ and report

$$H_\ell = -\sum_y p_\ell(y) \log_2 p_\ell(y), \quad S_\ell(y^*) = -\log_2 p_\ell(y^*), \quad P_\ell(C) = \sum_{y \in C} p_\ell(y).$$

Entropy need not decrease monotonically.

7.1 Conditional transformation variance

At fixed functor F, learned category C, and depth ℓ, estimate nuisance dispersion

$$\mathrm{Var}[\Delta r_\ell(X) \mid F, C]$$

alongside residual covariance/effective rank, target-logit variance, mean JS divergence to the (F, C) centroid, rule-consistent mass, surprisal, and entropy. A crossed decomposition separates F, C, $F \times C$, entity-within-C, and nuisance effects. The $F \times C$ interaction is evidence for type-sensitive dispatch only when C is behaviorally learned and held-out members generalize.

Entropy and conditional dispersion are complementary: several valid outputs can preserve high within-example entropy while nuisance-varying realizations converge on a stable rule-consistent distribution. Causal patching must therefore report effects on both mean rule signal and conditional variance.

A growing-context condition supplies increasing numbers of $F(a) \mapsto u, F(b) \mapsto u, \ldots$ demonstrations before querying $F(c)$. Demonstration count, distractor noise, and distance are crossed, yielding depth-by-examples-by-noise surfaces for accuracy, conditional JS dispersion, and between/within-rule SNR. Required diagnostics include within-(F, C) update variance, the between/within ratio, $F \times C$ interaction, held-out substitution invariance, entropy versus dispersion, and causal mean/variance effects. Variance is always across nuisance realizations at a fixed layer, never across layers.

7.2 Frozen downstream experiment set

E1 moves from lexical templates to substitution-invariant functors; E2 tests learned categories as soft types; E3 holds syntax fixed while changing type; E4 withholds arguments; E5 tests nested/chained composition; and E6 introduces exceptions and overrides. Interventions include layer/head skipping, equivalent-functor and category-state patching, counterfactual substitutions, and norm-matched random donors. Controls include shuffled types and functor mappings, lexical/frequency baselines, paraphrases, leakage audits, context length, undertrained/random checkpoints, and multiple seeds.

8 Dependency audit: historical and current

The original audit correctly blocked semantic dispatch because the legacy Paper 0.6 six-tree runs achieved only 1.4–15.3% held-out accuracy. That machine-readable decision is preserved as a historical stop record, not silently rewritten.

Subsequent Paper 0.6 work changes the landscape but does not fully open the gate. S3 nested attribute composition is competent (99.93% mean, 99.22% worst cell) and shows partial causal reuse. S1-v4 predicate extrapolation remains negative: parent, grandparent, root, and k-ancestor fail, while `isAncestor` is locally competent but seed/model-depth unstable. Thus bounded composition is a positive reference, but a learned semantic type system has not been established. The formal ladder may proceed; semantic type dispatch remains blocked until its own held-out member/type gate passes.

9 Interpretation and falsification

Possible outcomes are finite interpolation, capacity-dependent finite frontiers, stable systematic extrapolation over a tested range, or a separation between propositional proof and term/unification complexity. The strong negative outcome is scientifically central: larger fixed Transformers may move a bounded frontier without acquiring open-ended execution.

Rule-like interpretation is weakened when effects disappear on held-out symbols/combinations, reduce to lexical or position cues, ignore repeated-variable constraints, fail causal transfer, or appear equally in incompetent checkpoints. A mixed result—local, approximate, and context-dependent computation—is valid, provided its frontier is stated exactly.

10 Relationship to Papers 0.5, 0.6, and 0.9

Paper 0.5 supplies controlled predictive refinement and causal continuation equivalence. Paper 0.6 supplies competence gates, a bounded compositional positive, and a depth-general predicate negative. Paper 0.7 measures the formal one-shot execution boundary. Paper 0.9 begins from its existing manuscript and compares explicit iterative harnesses beyond that boundary; those results will not be mixed into the fixed-model claim.

11 Reproducibility and completion gate

Completion requires validated symbolic generators and canonical proofs/MGUs; disjoint identities where applicable; independent proof depth, term depth, arity, variables, branching, and distractors; unseen-template splits; three-seed central curves; training-budget and parameter controls; every requested measured frontier; and no mechanism claims outside competent cells. Pretrained results, if any, remain appendix-only. The historical eligibility decision is preserved at `results/eligibility_decision.json`. Paper 0.6 values are sourced from the tracked v2 gates and v4 predicate manifests/tables; Paper 0.5 dependence is sourced from its tracked causal replacement artifacts. The present v3 is an integrated preregistered protocol and does not report unrun formal experiments as findings.

The current formal-data checkpoint is generated by `cl.experiments.paper07_formal_phase` from `configs/paper07/formal_v1.json`. Its manifest records the dataset hash, validation outcome, row count, planned-run count, and the explicit flag `model_results=false`; the train, validation, and test JSONL files and phase-plan CSV are stored under `results/formal_v1`. This makes the boundary between validated experimental infrastructure and pending empirical evidence machine readable.

12 Conclusion

Paper 0.7 asks which exactly characterized computations a fixed Transformer can learn and execute, which resources move each finite frontier, and where one-shot execution fails so that iterative or symbolic alternatives merit controlled comparison. The fuzzy rule-like proposal remains intact as a downstream, competence-gated application. This ordering turns an earlier blocked semantic protocol into a coherent formal program without weakening its evidential standard.